CH - 01

Introduction to Software Engineering

Topics

- What does the term "software engineering" mean in the context of professional software development?
- Software engineering ethics - Describe the moral dilemma that faces the field of software development.
- Case studies - Four case study examples are provided.

Software engineering

- The software is one of the major factor that it is consider to determine the economics of developed country.
- Software is keep increasing used to control the system in all business and activity.
- In software engineering, the methods, theories, and tools are concerned for professional development.
- Investment on software development represents an important of Gross Nation Product.

Software costs

- Computer system costs are frequently driven by software costs. More often than not, the cost of hardware, software prices rise.
- Software maintenance costs are higher than development costs. Protection costs for structures with a long lifespan can also be many times higher than improvement expenses.
- Reasonable software program development involves software engineering.

Why Software project fail?

- **Not utilizing software engineering techniques:** It is pretty effortless to develop the software without the usage of software engineering strategies and approaches. Many groups have flowed into software program improvement as their merchandise and offerings have evolved. They do no longer use software engineering techniques in their every day work. Consequently, their software program is regularly extra cost and much less dependable than it need to be.

- **Expanding system complexity**: As current software program engineering methods assist to construct scaling system, greater complicated systems, and the needs change. Systems have to be constructed and delivered extra quickly; even extra complicated structures are required; structures have to have better abilities that had been formerly idea to be impossible.

Types of Software

Generic products

- Stand-alone systems that are advertised and made available to anyone who wants to buy them to use.
- Examples – PC software program such as software program for particular markets such as sale management for supermarket, general accounting programs, CAD software, and drawing tools. The specification for what the software should be able to do belongs to the developer, and it is up to him or her to determine whether to update the program.

Types of Software

Custom products

- Software that is ordered specifically by a client to satisfy their requirements.
- Examples – Bug tracking software, Taxi and private hire providers, Customized E-Commerce Solutions.
- The specification of what the software must do belongs to the buyer, who also decides on any necessary software modifications.

Important characteristics of good software

Product characteristic	Description
Efficiency	Software must now avoid wasting machine resources like memory and processing cycles. Therefore, processing speed, responsiveness, memory and storage usage, etc. comprise efficiency.
Maintainability	Maintainability changing wishes of customers. This is a fundamental attribute due to the fact software program exchange is an inevitable requirement of an altering commercial enterprise environment.
Dependability and security	Software dependability consists of vary of traits such as reliability, protection and safety. Dependable software program ought to no longer purpose bodily or financial harm in the tournament of gadget failure. Malicious customers need to no longer be capable to get right of entry to or harm the system.
Acceptability	The type of customers for whom software is intended should find it appealing. This represents that it should be practical, compatible, and intelligible with other frameworks that we employ.

Important characteristics of good software

Software engineering is a branch of engineering that deals with all activities of software program development, from early system definition stages to maintaining the system after it has been in use for some time.

- The engineering profession
  - Using brilliant theories and tactics to address problems while taking organizational and financial restrictions into consideration.
- Every aspect of software development
  - Not only a technological development method. Challenge management and the advancement of tools, processes, etc., are also important since they direct and assist the development of software.

Importance of software engineering

Humans and society are depending more and more on advanced software systems. We want to be able to create trustworthy and honest institutions fast and cheaply.

Instead of merely writing the applications as if it were a private programming project, it is often more cost-effective in the long term to employ software engineering techniques and techniques for software structures. The majority of costs for most system types are associated with changing the software after it has been in use for a while.

Software process activities

- **Software specification**: The software program that is to be generated and the restrictions on its functioning are outlined in the software specification, clients, and engineers.
- **Software development**: The process of designing and coding a software program..
- **Software validation:** The program is examined to make sure it satisfies the needs of the user.
- **Software evolution**: A program's code is changed to replicate changing user and market demands.

General factors that have an effect on software

- **Heterogeneity**: Networks that involve various types of pc and mobile devices are increasingly expected to function as assigned structures.
- **Business and social change**: They are changing quite swiftly as emerging economies become more powerful and current technology is more widely available. They desire the ability to exchange their current software and surprise enhance new software.
- **Security and trust**: Because software is ingrained in every aspect of our lives, it is essential that we have faith in it.
- **Scale**: A wide range of scales must be considered while developing software, from extremely tiny embedded structures in wearable or portable devices to cloud-based systems and Internet-scale that support a global community.

Software engineering distinctiveness

There are many distinct sorts of software program device and there is no usual module of software methods that is relevant to all of these functional.

The software program engineering techniques and equipment used rely on the kind of utility being developed, the necessities of the client and the heritage of the improvement team.

Application types

- **Stand-alone**: It is the software programs that run alone on a private devices, like a PC. They are no longer need network connectivity and have all required features.
- **Interactive transaction-based functions**: Applications that run on a remote device and are accessed by client from their own devices or PC are interactive transaction-based functions. These include e-commerce apps and other internet-related uses.
- **Embedded control**: It is the control systems and software-based management for hardware components. In terms of numbers, embedded structures are perhaps more prevalent than other types of systems.
- **Batch processing**: It is the business systems made to deal with huge amounts of information in batches. Massive amounts of user input are processed to provide related outcomes.

Application types

- **Entertainment**: It is the application that is mainly intended for personal usage and are meant to amuse the user.
- **Modelling and simulation**: It is created by scientists and engineers to simulate physical processes or situations that involve numerous, unrelated, interacting components.
- **Data collection**: It collects data from their environment using a variety of sensor devices then send that data to various processing structures are known as data collecting systems.
- **Systems of systems**: These systems are made up of numerous various software application systems.

Basics of software engineering

Regardless of the enhancement methods employed, the following fundamental notions to all types of software application systems:-

- Systems should be designed with a controlled and clear improvement process. Of course, exceptional software requires outstanding strategies.
- For all systems, dependability and overall performance are essential.
- It's necessary to comprehend and manage the software program requirements (what the software program must perform).
- Whenever possible, you should repurpose existing software rather than creating it from scratch.

Software engineering for mobile and the internet

Today, software may be executed on the Web and mobile applications, and businesses are rapidly developing web-based structures as an alternative to local systems.

Web services make it possible to access utility performance over the Internet.

A method of supplying laptop services called "cloud computing" allows for distant application execution on the "cloud."

Instead of purchasing software, users pay usage-based fees.

Internet-based software engineering

The fundamental concepts of software engineering that were previously discussed apply to internet-based systems just as they do to other types of systems, despite the fact that they have complicated distributed architectures.

Internet-based software is subject to the same fundamental principles of software engineering as other kinds of software systems.

Internet software engineering

- **Incremental and agile development**: Systems that use the internet must be developed and launched gradually. The fact that it is impracticable to predetermine every requirement for such designs is now well understood.
- **Software reuse**: The predominate approach to creating Internet-based systems is software reuse. When constructing these systems, you take into account how to assemble them from already-existing software program elements and systems.
- **Service-oriented systems**: It is also possible to construct software in cases where the individual software modules are web services.
- **Rich interfaces:** Rich interfaces are now possible in web browsers because to the development of software interface design technologies like AJAX and HTML5.

Ethics in software engineering

More responsibilities than only the application of technical skills are involved in software engineering.

If software engineers want to be considered as professionals, they must do honestly and responsibly from an ethical standpoint.

Beyond simply observing the law, ethical behaviour also involves adhering to a set of ethically sound principles.

Expert responsibility issues

- **Competence**: Software engineers are no longer required to overstate their level of ability. They must now refrain from taking delivery of work that is outside the scope of their expertise.
- **Confidentiality**: Software engineers should typically protect the confidentially of their employers or clients. Regardless of whether or not a formal confidentiality agreement has been formed.
- **Computer misuse**: Developers are no longer allowed to exploit their technical expertise to improperly utilize other people's computers. Computer abuse ranges from the incredibly minor (playing games on a work computer, for example) to the incredibly serious (dissemination of viruses).
- **Intellectual property rights**: Software developers should be aware of regional rules pertaining to the usage of intellectual property, such as copyright and patents. To protect the intellectual property of employers and buyers, they must exercise vigilance.

Code of Ethics “ACM/IEEE”

The expert societies in the US have cooperated to produce a code of moral practice.

Members of these organizations signal up to the code of exercise when they join.

The Code incorporates eight Principles associated to the behaviour of and choices made through expert software program engineers, along with educators, practitioners, supervisors, managers, and coverage makers, as nicely as college students and trainees of the profession.

Rationale for the code of ethics

“Computers have a central and developing position in commerce, industry, government, medicine, education, leisure and society at large. Software engineers are these who make a contribution through direct participation or with the aid of teaching, to the analysis, specification, design, development, certification, preservation and checking out of software program systems.”

“Because of their roles in creating software program systems, software program engineers have widespread possibilities to do precise or motive harm, to allow others to do suitable or motive harm, or to affect others to do desirable or motive harm. To ensure, as tons as possible, that their efforts will be used for good, software program engineers need to commit themselves to making software program engineering a really helpful and revered profession.”

The ACM/IEEE Code of Ethics

Code of Ethics and Professional Practice for Software Engineering
Software Engineering Ethics and Professional Practices Joint Task Force of ACM/IEEE

PREAMBLE
The code's brief model lists goals at an excessively high level of abstraction; the full model's clauses, on the other hand, give instances and highlight key aspects of how these goals affect our behavior as software program engineering professionals. Without the aspirations, the tiny print may become monotonous and legalistic; without the specifics, the aspirations may become overly lofty but meaningless; when combined, the aspirations and the small print form a coherent code.

The analysis, specification, design, development, testing, and maintenance of software will be made into a highly valuable and respected career by software engineers. Software program developers must abide by the following eight principles in order to uphold their commitment to the public's health, welfare, and protection.

1. PUBLIC - Software engineers must always behave in the best interests of the general public.
2. CUSTOMER AND EMPLOYER - Software engineers must act in the best interests of their customer and employer while also upholding the general welfare.
3. PRODUCT - Software developers must make sure that their products and any related modifications adhere to the highest professional standards.
4. JUDGMENT - Software engineers must maintain independence and honesty in their professional judgment.
5. MANAGEMENT - Managers and leaders in the field of software engineering must support and advance a moral approach to the supervision of software program development and upkeep.
6. ROFESSION - Software engineers must uphold the integrity and popularity of their profession while keeping it in line with the general good.
7. COWORKERS - Software developers must be trustworthy and encouraging to one another.
8. SELF - Software engineers must engage in lifelong learning pertaining to the practice of their profession and must advocate for a moral approach to the practice of their trade.

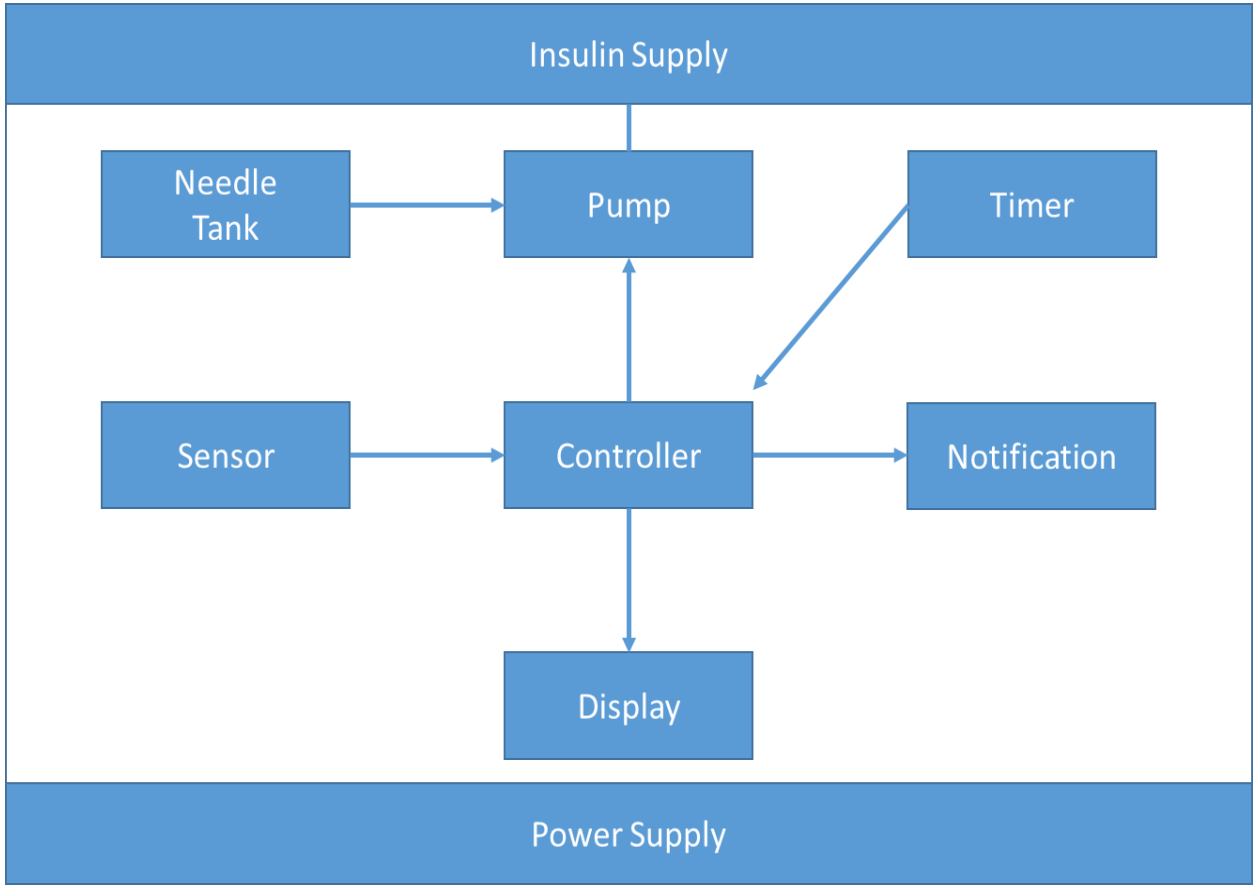
Ethical dilemmas

- Perceptual disagreement with upper management's policies.
- Your business releases a safety-critical machine without stopping the system's checking out, which is unethical behaviour.
- Involvement in the development of nuclear or military weapons systems.

Case studies#1:
Insulin pump for personal used

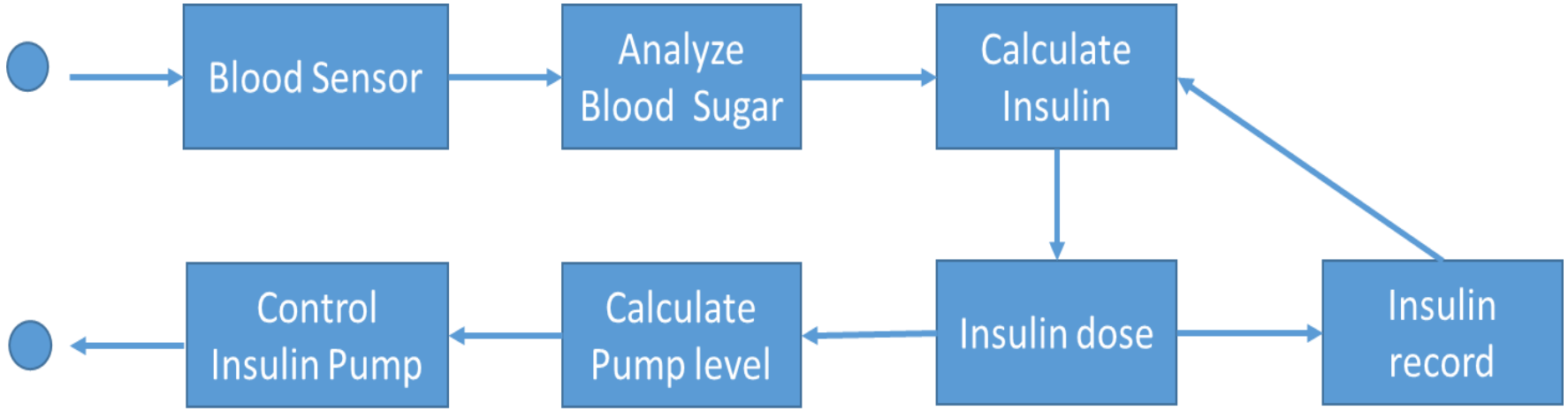
- Gathers data from a blood sugar sensor and determines how much insulin needs to be delivered.
- A calculation that is entirely based on the value at which blood sugar levels fluctuate.
- Transmits indicators to a pump to inject the appropriate amount of insulin for the patient’s condition.
- Safety-critical equipment since low blood sugar levels be able to cause mental dysfunction, coma, and death; excessive blood sugar levels be able to cause long-term consequences such kidney and eye damage.

Case studies#1:
Insulin pump for personal used



Hardware structure of the Insulin pump

Case studies#1:
Insulin pump for personal used



The process of the insulin pump

Case studies#1:
Insulin pump for personal used

Important high-level requirements for Insulin pump

- The device should be handy to supply right insulin level when required.
- The device should function supply the right insulin level to counteract the present stage of blood sugar with high reliable.
- The machine ought to consequently be planed and developed to make sure that the device continually fits these system requirements.

Case studies#2: (Mentcare)
A mental health patient information system

- Mentcare is an informational database that compiles statistics on individuals with mental health issues and the treatments they have received from the therapy.
- Most mental health patients do no longer need devoted medical institution therapy however want to attend professional frequently the place they can see a physician who has distinctive expertise of their problems.
- These clinics are now run outside of hospitals to make it convenient for patients to visit. Additionally, neighbourhood medical offices or community centers may host them.
- An information system called Mentcare is designed specifically to operate in clinics.

Case studies#2: (Mentcare)
A mental health patient information system

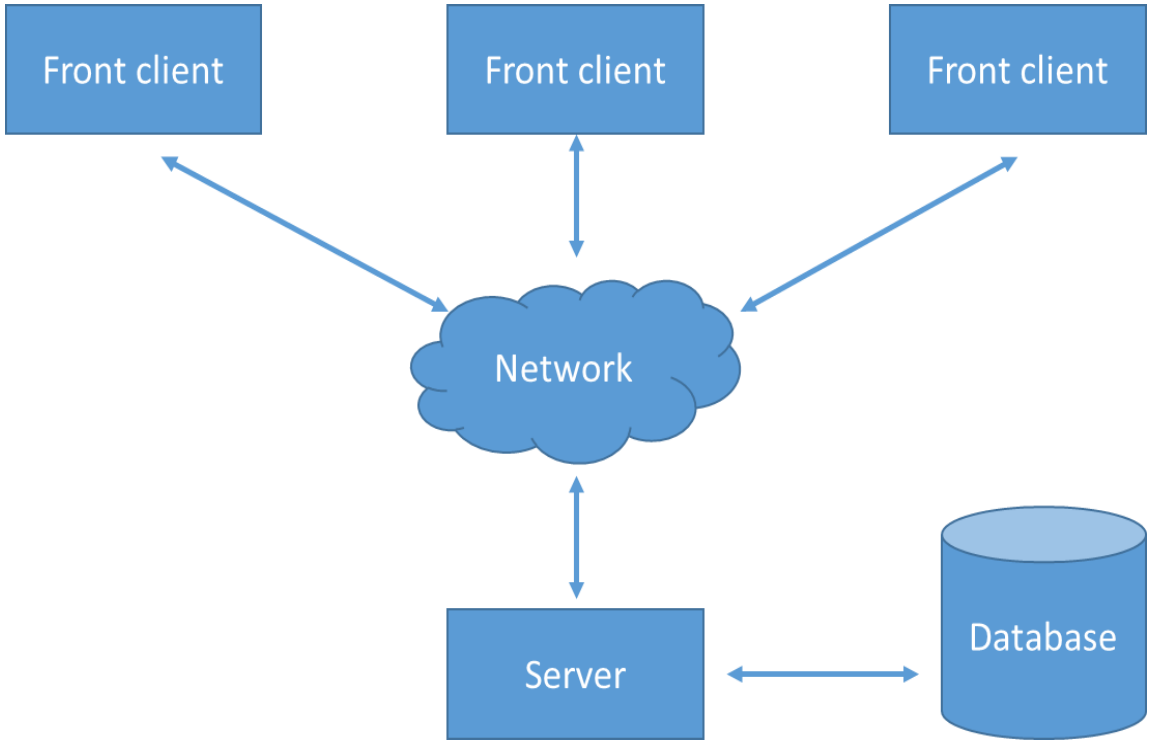
- It records patient data in a centralized database but has also been built to operate on a lab computer so that it may be used via web browser without needing network connectivity.
- The system store the patient record in the database during the available of the secure network connectivity, but the user can download the copies of patient record while the network is down.

Case studies#2: (Mentcare)
A mental health patient information system

Mentcare goals

- To generate administration data that lets in health services managers to determine overall performance in opposition to nearby and authority's objectives.
- To serve medical staff with real-timed data to aid the therapy of sufferers.

Case studies#2: (Mentcare)
A mental health patient information system



The system architecture of the Mentcare

Case studies#2: (Mentcare)
A mental health patient information system

The Mentcare system's main features

- **Patient monitoring**: The information of the patients who are concerned about their treatment and offers alerts if real difficulties are found is keep monitoring.
- **Individual care administration**: Clinicians can input data for patients, change system statistics, see patient histories, etc. The tool aids in information brief so that doctors can quickly analyse the important issues and provide remedies.
- **Administrative and management reporting**: At each clinic, the system can generate monthly administration and management reports that detail the vast variety of patients treatment.

Case studies#2: (Mentcare)
A mental health patient information system

Important issues in Mentcare system

- **Privacy**: Personal information about impacted individuals must always be kept private and never disclosed to anybody save authorized staff members and the impacted individual.
- **Safety**: Some mental diseases lead to suicidal thoughts or interference with others. The system must, wherever possible, alert medical professionals to people who are potentially dangerous or suicidal. The system must be accessible at all times; otherwise, security may be compromised and it might not be possible to deliver patients the right medication.

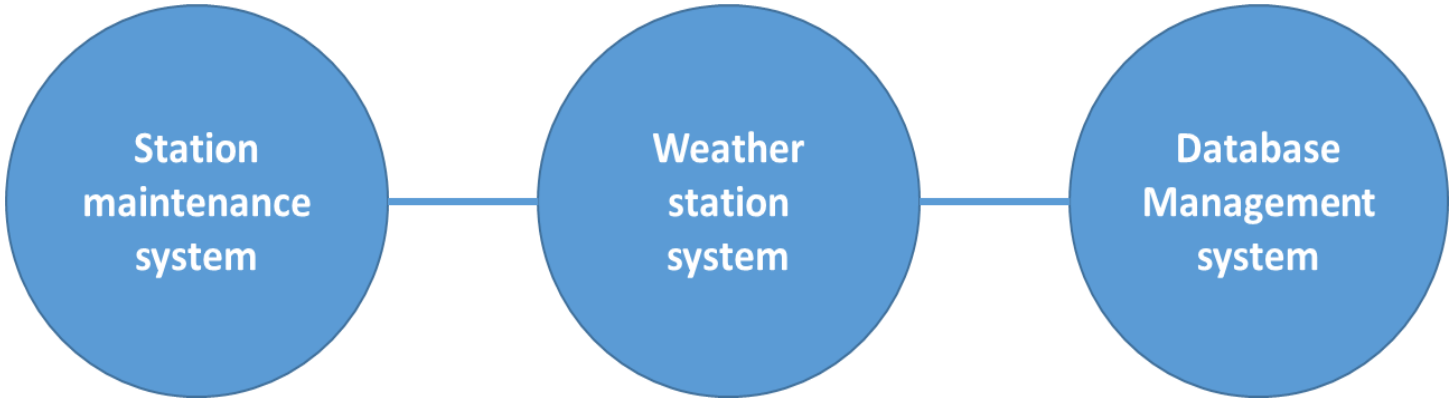
Case studies#3: Climate station

The authorities of a United States of America with giant areas of barren region decides to install a number of hundred climate stations in faraway areas.

Climate stations compile data from a variety of sensors that gauge temperature, humidity, pressure, sunlight, precipitation, wind direction, and wind speed.

The climate station is made up of a number of sensors that track several aspects of the climate over a 24-hour period, including the wind direction/speed, barometric pressure, air and floor temperatures, and rainfall. A software program machine that controls the records gathered from the devices and periodically takes parameter measurements is used to manage each of these devices.

Case studies#3: Climate station



The setting of the climate station network

Case studies#3: Climate station

Climate information system

The equipment called a climate station serves as a climatic information system. It is the statistics administration and archiving system.

This is in charge of gathering weather data, performing some basic information processing, and sending the result to the information administration system.

This system gathers data from all weather stations, houses the processing and evaluation of statistics, and records the information.

The maintenance system will link to all climate stations via satellite TV for PC to show the status of these structures and produce reviews of issues.

Case studies#3: Climate station

Additional features of the software

- Keep an eye on the hardware connection, energy, and gadgets, and report any issues to the administration system.
- Manage the device power, making sure that the battery's powers are charged whenever the environmental conditions allowance but also that turbines are turned off in perhaps unfavorable weather situations, such too much wind.
- Support for various reconfigurations, including those in which outdated software components are supplied with newer versions and backup devices are replaced in to take their place in the event of a device failure.

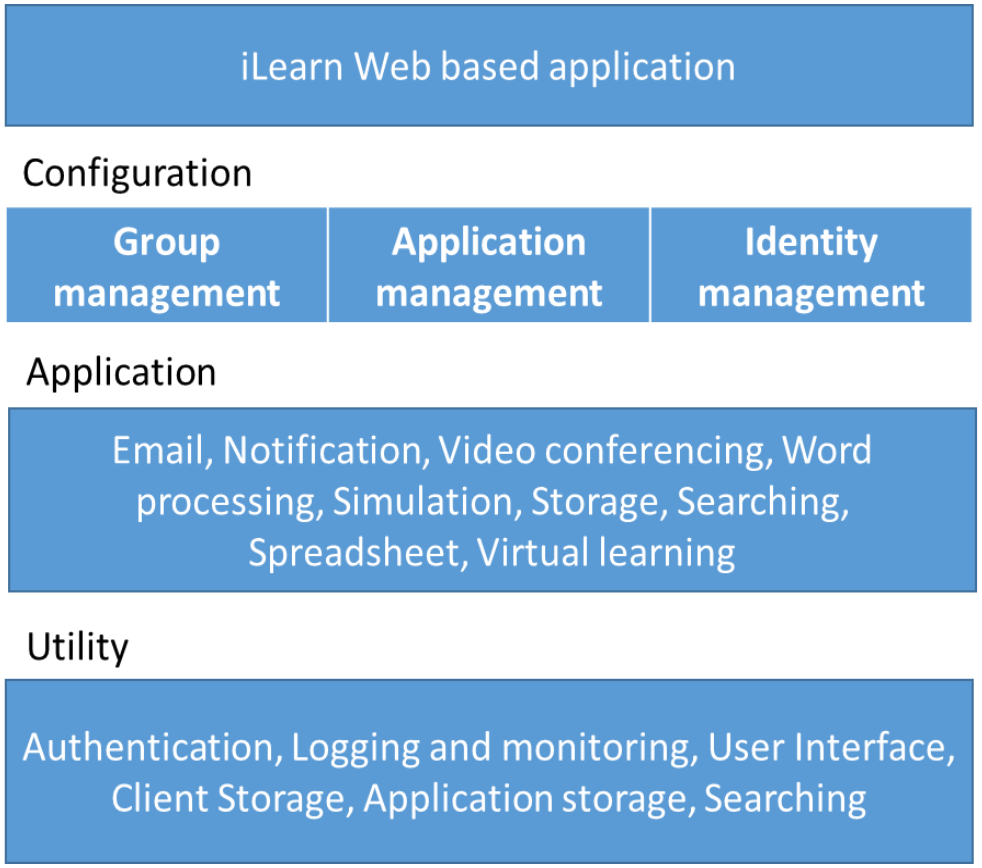
Case studies#4:
A digital classroom (iLearn)

iLearn is a system framework that a number of general and specifically created learning tools can also be included, along with a number of purposes catered to the needs of system newcomers.

The resources included in each A digital classroom version are picked by the teacher and the students to meet their individual needs.

These could include common activities like spreadsheets, learning about administrative features like a Virtual Learning Environment (VLE) to conduct assignment / homework submission and evaluation, simulation, video games, and quiz.

Case studies#4:
A digital classroom (iLearn)



iLearn system architecture

Case studies#4:
A digital classroom (iLearn)

Service-oriented systems of iLearn

- The device is a service-oriented system, meaning that every component is seen as a changeable module.
- This enables the device to be updated gradually as new versions become applicable.
- Additionally, it leads to quickly customize the device to create different environments for certain agencies, such as very young student who are illiterate, senior citizens, etc.

Case studies#4:
A digital classroom (iLearn)

iLearn's functions

- **Utility services**, which bring core application-independent performance and also be accessed through other system offers.
- **Application services** that bring specific functions like conferencing, image sharing, email, etc. and access to special educational content like historical documents or scientific movies.
- **Configuration services** that describe how offerings are shared between kids, teachers, and their parents as well as how to adjust the environment with a specific set of software products.

Case studies#4:
A digital classroom (iLearn)

Service integration for iLearn

- Integrated services are products that pass an Application Programming Interface (API) and that may be used to access other products using that API. As a result, direct verbal communication between services is possible.
- Independent services are offerings that may be used by web browser and operate independent from other services. Data can be transferred between services using consumer actions like copying & pasting; each unbiased provider may additionally need re-authentication.

Summary

- Software engineering is a subpart of engineering involves with every aspect of creating a software program.
- Maintainability, dependability, security, efficacy, and acceptability are crucial characteristics of software program products.
- All software development processes involve activities of specification, analysis, design, development, testing, implementation, and evolution.
- All forms of system development can benefit from understanding the fundamental principles of software engineering.

Summary

- There are numerous different system types, as well as specialized methodologies and techniques for developing software programs.
- All types of software program systems can benefit from understanding the core concepts of software engineering.
- Software developers have obligations to society and engineering professionals. They shouldn't, without a doubt, be involved in technological matters.

Exercise Questions

What characteristics define quality software?

What core activities make up software engineering?

How much does software engineering cost?